IQB · METHODEN-TEAM

eatPrepTBA

Vom Testcenter zum Skalierungsdatensatz

Karoline Sachse · 26.10.2026

Aufbau des Pakets · Datenaufbereitung · Logdaten · Codebücher



Struktur

Thema	Inhalt
1 · Paketaufbau und Datenabruf	IQB-Studio und Testcenter; Daten beziehen; JSON und R-Objekte verstehen
2 · Datenaufbereitung & Kodierung	Rechenaufgabe und kleine Studie; manuelle Codes; Missing-Regeln; Kennwerte und Skalierungsdatensatz
3 · Logdaten	Ereignisse überblicken, Zeiten und Fokusverluste auswerten
4 · Codebücher und Kodiermanuale	Codebücher als Excel/CSV; DOCX direkt aus IQB-Studio

Materialien und Skripte

Synthetische Beispieldaten ermöglichen alle lokalen Schritte ohne eigenen Zugang. Das Handout enthält Code, Ausgaben und Erläuterungen.

Datei in `R/`	Verwendung
`00_installation.R`	Pakete installieren, laden und Dateien prüfen
`01_datenabruf.R`	Daten aus IQB-Studio und Testcenter beziehen
`02_daten_verstehen.R`	JSON, vorhandene Codes und Rechenaufgabe
`03_datenaufbereitung.R`	Vollständige Aufbereitung der kleinen Studie
`04_logs.R`	Ereignisse, Zeiten und Fokusverluste
`05_kodiermanuale.R`	Lokales Codebuch als Excel und CSV
`06_kodiermanuale_studio.R`	Codebuch und DOCX aus IQB-Studio

Die Skripte sind linear ausführbar und dienen als Vorlage für eigene Daten.

1 · Paketaufbau und Datenabruf

eatPrepTBA verbindet Antworten, Aufgabeninformationen und Testdesign.

Zuerst: Paket und Datenquellen. Anschließend: Datenabruf und die dabei entstehenden R-Objekte.

Das Paket eatPrepTBA



Daten abrufen → Antworten aufbereiten und kodieren → Auswertungen vorbereiten

Zusätzlich: Logdaten auswerten und Codebücher erstellen.

- Paketdokumentation: Funktionshilfe und Vignetten.
- GitHub-Repository: Quellcode und Entwicklung.
- Issues: Fehler oder Erweiterungswünsche melden.

Für einen Fehlerbericht: Paketversion, kleiner reproduzierbarer Code, Fehlermeldung und erwartetes Ergebnis; anonymisierte oder synthetische Daten verwenden.

Wie kommuniziert R mit den Webanwendungen?

Die **API** ist die Schnittstelle, über die R Daten aus IQB-Studio oder Testcenter anfordert.



- **eatPrepTBA** wählt den passenden Endpunkt, etwa die Units eines Workspaces. Die Anmeldung liefert einen Sitzungstoken für weitere Anfragen.
- httr2 sendet die HTTP-Anfragen und empfängt die Antworten.
- **eatPrepTBA** bereitet die Antwort für R auf: JSON wird zunächst zu Listen, daraus entstehen Tabellen; einige Spalten bleiben verschachtelt.

API-Einführung: Kommunikation mit IQB-Studio und Testcenter

Datenquellen und Einlesefunktionen

Quelle	Benötigte Inhalte	Funktionen
IQB-Studio	Variablen, Kodierschema, Item-Verknüpfungen, Aufgabenpositionen	`get_units()`
Testcenter: Antworten	gespeicherte Eingaben	`get_responses()` oder `read_responses()`
Testcenter: Testdesign	vorgesehene Personen, Booklets und Reihenfolge	`get_design()`
Testcenter: Logs	Ereignisse während der Durchführung	`get_logs()` oder `read_logs()`

`get_*()` ruft Daten über die **API** ab, die Schnittstelle zur Anwendung. `read_*()` liest vorhandene Exportdateien. Danach liegen R-Objekte vor.

Antwort- und Logexporte ersetzen weder die Aufgabeninformationen noch das vollständige Testdesign.

Eigene Zugänge: Einstellungen zuerst anpassen

Die Version steht auf der IQB-Studio-Anmeldeseite. Die Workspace-IDs 1300 und 35 sind Beispiele und werden durch die eigenen IDs ersetzt.

```r
library(eatPrepTBA)

studio_version <- "21.0.0" # genaue Version der Anmeldeseite übernehmen
studio_workspace_id <- 1300 # Beispiel; eigenen Workspace eintragen
testcenter_url <- "https://iqb-testcenter2.de/"
testcenter_workspace_id <- 35 # Beispiel; eigenen Workspace eintragen
```

Skript: 01_datenabruf.R

Was bedeutet der Testmodus?

`get_design()` berücksichtigt standardmäßig Testpersonen-Logins mit `mode = "run-hot-return"`. Deshalb benötigt der Beispielaufruf kein zusätzliches Argument.

`run-hot-return` bedeutet: Antworten und Logs speichern, Zeit- und Navigationsbeschränkungen anwenden und bei erneuter Anmeldung die bisherige Sitzung fortsetzen.

Bei einem anderen Testdurchführungsmodus `mode` ausdrücklich angeben. Der Filter betrifft die Testpersonen im Design, nicht die Anmeldung des Administrationskontos.

Der Modus filtert das Design; er stellt nicht nachträglich die Testdurchführung um.

Testcenter-Hilfe zu den Testmodi

IQB-Studio: Aufgabeninformationen laden

Ohne `base_url` verwendet `login_studio()` die Installation **www.iqb-studio.de**. Danach den Workspace mit den Aufgaben auswählen.

```r
dir.create("data_eigen", showWarnings = FALSE)
```

```r
studio_login <- login_studio(app_version = studio_version, keyring = TRUE)
studio <- access_workspace(studio_login, ws_id = studio_workspace_id)

units <- get_units(studio, metadata = TRUE, unit_definition = TRUE)
saveRDS(units, "data_eigen/units.rds")
```

`metadata = TRUE` bereitet die Metadaten auf. `unit_definition = TRUE` lädt zusätzlich die technische Aufgabendefinition.

Vorlage für eigene Zugänge: 01_datenabruf.R

Anmeldung: Passwortspeicher und Fehlermeldungen

`keyring = TRUE` speichert die Zugangsdaten lokal im Passwortspeicher des Betriebssystems und verwendet sie bei weiteren Anmeldungen erneut.

Beobachtung	Nächster Schritt
HTTP 401 Unauthorized	Installation und Zugangsdaten prüfen. Mit `keyring = FALSE` einmal frisch eingeben.
Gespeicherte Zugangsdaten sind veraltet	Einmal `keyring = TRUE, change_key = TRUE` verwenden, um sie zu ersetzen.
HTTP 521	`app_version` auf die genaue Version der Anmeldeseite setzen, einschließlich eines möglichen Beta-Zusatzes.

Nach einem IQB-Studio-Update erneut anmelden und das Workspace-Objekt neu erstellen.

Die auskommentierten Aufrufe stehen in R/01_datenabruf.R.

Aufgabendefinition und Unit-Tabelle

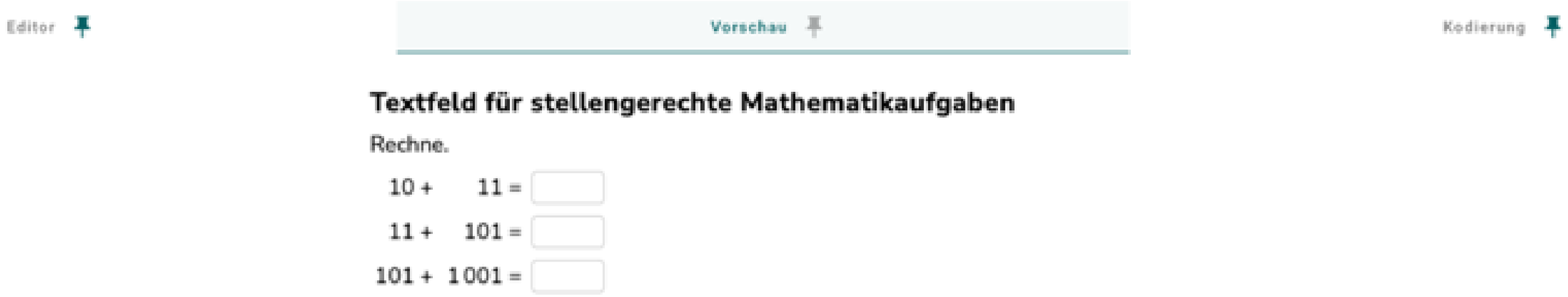
`unit_definition` enthält die technische Definition als JSON: Seiten, Abschnitte, Texte und Medien, Antwortfelder sowie deren Anordnung und Einstellungen. Die genaue Struktur hängt vom Editor ab.

Das äußere Objekt `units` führt diese Definition mit weiteren Informationen zusammen:

Gruppe	Beispiele für Spalten
Workspace und Unit	`ws_id`, `ws_label`, `unit_id`, `unit_key`, `unit_label`
Variablen und Kodierung	`unit_variables`, `coding_scheme`, `player`, `editor`, `schemer`
Metadaten und Items	`unit_profiles`, `items_list`, `items_profiles`
Definition und Positionen	`unit_definition`, `variable_pages`
Bearbeitungsstand	`state_id`, `last_change_definition`, `last_change_scheme`

Der vollständige Feldkatalog steht im Handout. `names(units)` zeigt die tatsächlich gelieferten Spalten.

Eine Unit in IQB-Studio: drei Eingabefelder



Die gesamte Aufgabe ist eine **Unit**. Die drei Eingabefelder heißen `01a`, `01b` und `01c`.

Das synthetische Beispiel verwendet dieselben Variablennamen und die zentralen Kodierregeln.

Dasselbe Beispiel lokal in R laden

Die Daten wurden passend zum Screenshot synthetisch aufgebaut. Es sind keine aufgezeichneten Serverantworten.

```r
library(eatPrepTBA) # eatAutoCode nicht separat laden.
library(dplyr)
library(tidyr)

pc_units <- readRDS("data/demo-pc-units.rds")
pc_responses <- read_responses("data/demo-pc-responses.csv")

select(pc_units, unit_key, unit_label)
select(pc_responses, login_name, unit_key, response_progress)
```

Vier Antwortmuster zeigen später volle, teilweise und keine Punktzahl.

Skript: 02_daten_verstehen.R

Die Unit-Tabelle und ihre Spalten

```
pc_units |>
  select(unit_key, unit_label, coding_scheme, unit_variables, items_list) |>
  print(width = 90)
names(pc_units)
```

```
# A tibble: 1 × 5
  unit_key   unit_label
  <chr>      <chr>
1 PC_RECHNEN Rechenpaeckchen
  coding_scheme                                          unit_variables items_list
  <chr>                                                  <list>         <list>
1 "{\"variableCodings\":[{\"id\":\"01a\",\"alias\":\"01… <tibble>       <tibble>
```

```
[1] "ws_id"          "ws_label"       "unit_id"        "unit_key"
[5] "unit_label"     "coding_scheme"  "unit_variables" "items_list"
[9] "unit_codes"
```

coding_scheme enthält die JSON-Regeln. unit_variables und items_list sind verschachtelte Tabellen; `names()` zeigt alle verfügbaren Spalten.

Die Tabellen innerhalb einer Unit öffnen

`[[1]]` öffnet die Tabelle für die erste Unit innerhalb einer Listenspalte.

```
1 print(pc_units$unit_variables[[1]] |> select(variable_id, variable_type))
2 print(pc_units$items_list[[1]] |> select(variable_id, item_id))
```

```
# A tibble: 3 × 2
  variable_id variable_type
  <chr>       <chr>
1 01a         string
2 01b         string
3 01c         string
```

```
# A tibble: 1 × 2
  variable_id item_id
  <chr>       <chr>
1 01          RECHNEN_01
```

In diesem Beispiel wird **01** als Item ausgewertet. Die drei Ausgangsvariablen sind keine zusätzlichen Items.

Kodierschema vorhanden – wozu `add_coding_scheme()`?

`get_units()` lädt das Kodierschema bereits in der Spalte `coding_scheme`.

`add_coding_scheme()` macht daraus zusätzlich die verschachtelte Tabelle `unit_codes`. Darin lassen sich Variablen und Kodierregeln gezielt ansehen.

```
1 units <- add_coding_scheme(units) # coding_scheme wird zu unit_codes aufbereitet.
2 View(units)
```

Dieser Schritt ist zum Nachsehen nützlich. `code_responses()` mit `prepare = TRUE` bereitet das Kodierschema bei Bedarf selbst auf.

`View(units)` öffnet die vollständige Tabelle in RStudio. `units$unit_codes[[1]]` zeigt die Kodierinformationen der ersten Unit.

Testcenter: Antworten abrufen

Den Workspace der Durchführung auswählen und die Antworten im Rohformat sichern.

```r
testcenter_login <- login_testcenter(base_url = testcenter_url)
testcenter <- access_workspace(testcenter_login,
                               ws_id = testcenter_workspace_id)
```

```r
responses <- get_responses(testcenter)
saveRDS(responses, "data_eigen/responses.rds")

# Alternative, wenn bereits ein Antwortexport vorliegt:
# responses <- read_responses("data_eigen/Responses.csv")
# saveRDS(responses, "data_eigen/responses.rds")
```

Skript: 01_datenabruf.R

Im Antwortexport steckt JSON

Die Datei ist eine Tabelle. Eine ihrer Zellen enthält die Antworten einer Person auf die Unit als JSON.

```
# A tibble: 2 × 3
  login_name unit_key   responses
  <chr>      <chr>      <chr>
1 D1         PC_RECHNEN "[{\"id\":\"01a\",\"value\":\"21\",\"status\":\"VALUE_CHA…
2 D2         PC_RECHNEN "[{\"id\":\"01a\",\"value\":\"21\",\"status\":\"VALUE_CHA…
```

Im JSON stehen beispielsweise `id`, `status` und `value`. Der Wert 21 ist zunächst eine Eingabe, noch kein Punktwert.

Eine gespeicherte Antwort genauer ansehen

`{ ... }` beschreibt einen Eintrag mit benannten Feldern; `[ ... ]` eine Folge von Einträgen.

```
1 cat(jsonlite::prettify(pc_responses$responses[[1]]))
```


```
[
    {
        "id": "01a",
        "value": "21",
        "status": "VALUE_CHANGED"
    },
    {
        "id": "01b",
        "value": "112",
        "status": "VALUE_CHANGED"
    },
    {
        "id": "01c",
        "value": "1102",
        "status": "VALUE_CHANGED"
    }
]
```

Welche Ausgangsdaten liegen vor?

Ausgangslage	Passender Einstieg
Rohantworten und passende Kodierschemata, z. B. eine reguläre Studie	`code_responses()` kodiert nach diesen Regeln
Rohantworten ohne Kodierschema, z. B. bei StarS	`prepare_responses()` macht die Werte zugänglich
Bereits gespeicherte Codes und Scores	`prepare_coded()` liest diese Kodierungen aus

Die folgenden Beispiele zeigen beide Wege. Bei vorhandenen Codes müssen Herkunft, Bedeutung und Vollständigkeit vor der weiteren Analyse geklärt sein.

Rohantworten mit `prepare_responses()` entpacken

Aus dem verschachtelten JSON wird eine Zeile je gespeicherter Variable.

```r
raw_values <- prepare_responses(pc_responses)
select(raw_values, login_name, variable_id, value, variable_status)
```

```
# A tibble: 3 × 4
  login_name variable_id value      variable_status
  <chr>      <chr>       <list>     <chr>
1 D1         01a         <list [1]> VALUE_CHANGED
2 D1         01b         <list [1]> VALUE_CHANGED
3 D1         01c         <list [1]> VALUE_CHANGED
```

`value` ist zunächst eine Listenspalte: Manche Antwortformate enthalten mehrere Werte.

Skript: 02_daten_verstehen.R

Einzelne Antwortwerte sichtbar machen

Im Beispiel enthält jedes Feld genau einen Wert. `unnest_longer()` löst die Listenspalte auf.

```r
raw_values <- unnest_longer(raw_values, value, keep_empty = TRUE)
select(raw_values, login_name, variable_id, value, variable_status)
```

```
# A tibble: 3 × 4
  login_name variable_id value variable_status
  <chr>      <chr>       <chr> <chr>
1 D1         01a         21    VALUE_CHANGED
2 D1         01b         112   VALUE_CHANGED
3 D1         01c         1102  VALUE_CHANGED
```

Skript: 02_daten_verstehen.R

Vorhandene Kodierungen mit `prepare_coded()` lesen

Beispiel für den Weg ohne eigenes Kodierschema, etwa bei StarS: bereits gespeicherte Codes und Scores auslesen.

```r
pc_stored <- read_responses("data/demo-stored-codes.csv")
stored_codes <- prepare_coded(pc_stored)
select(stored_codes, login_name, variable_id, value, code_id, code_score)
```

```
# A tibble: 4 × 5
  login_name variable_id value code_id code_score
  <chr>      <chr>       <chr>   <int>      <int>
1 D1         01          3           1          2
2 D2         01          2           2          1
3 D3         01          1           0          0
4 D4         01          0           0          0
```

Skript: 02_daten_verstehen.R

Das vollständige Testdesign abrufen

Das vollständige Testdesign beschreibt die vorgesehenen Personen, Booklets und Aufgabenreihenfolgen.

```r
design <- get_design(testcenter, units = units)
saveRDS(design, "data_eigen/design.rds")
```

Dazu müssen Testpersonen-XML und zugehörige Booklet-XMLs im Workspace vorliegen. Ein Antwort-CSV allein liefert das vollständige Design nicht.

Skript: 01_datenabruf.R

2 · Datenaufbereitung & Kodierung

Beispiel 1: Rechenaufgabe — Drei Eingabefelder und ein Item mit Teilpunkten.

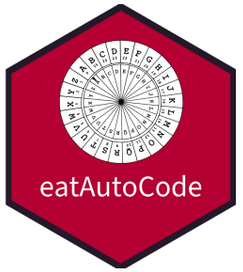
Beispiel 2: Kleine Studie — Antworten, manuelle Codes und Testdesign bis zum Skalierungsdatensatz.

eatPrepTBA und eatAutoCode arbeiten zusammen



eatPrepTBA verbindet Antworten, IQB-Studio-Metadaten, manuelle Codes und Testdesign.

`code_responses()` übergibt Antworten und passende Kodierschemata an eatAutoCode.



eatAutoCode macht den IQB-Autocoder in R nutzbar.

Es führt die Kodierung aus und gibt die Ergebnisse an eatPrepTBA zurück.

Im Workshop `library(eatPrepTBA)` laden. Ein zusätzliches `library(eatAutoCode)` würde dessen gleichnamige Funktion `code_responses()` in den Vordergrund stellen.

Was macht der IQB-Autocoder?

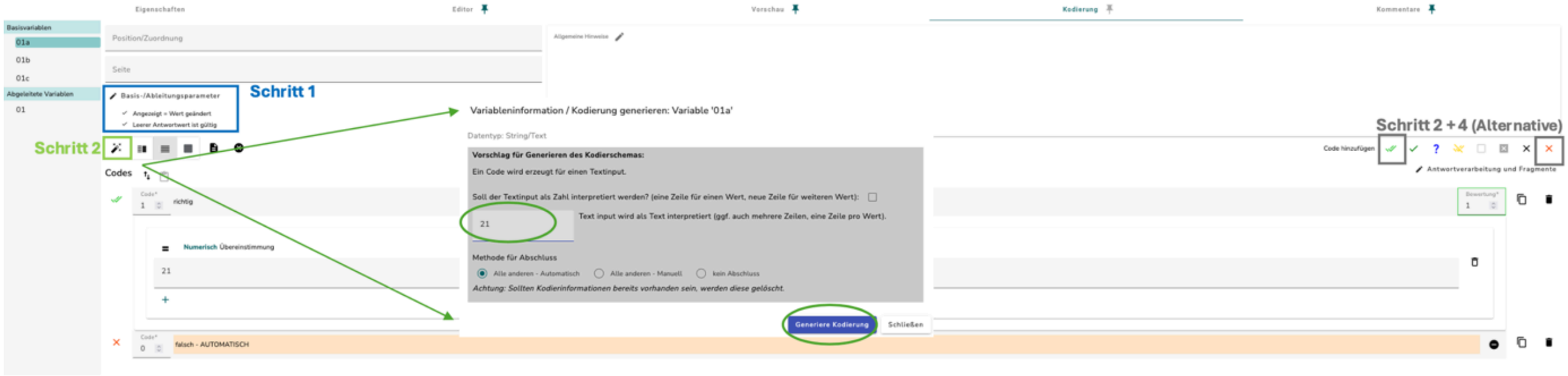
Der Autocoder stammt aus dem JavaScript-Paket **@iqb/responses**.

1. Eingang: Variablenkennungen und gespeicherte Antworten.
2. Das **Kodierschema** legt Regeln, Codes, Scores und Ableitungen fest.
3. Der Autocoder wendet die Regeln an und berechnet abhängige Variablen.
4. Ergebnis: `value`, `code_id`, `code_score`, `code_status`.

eatAutoCode führt die Kodierung lokal aus. eatPrepTBA verbindet das Ergebnis mit den benötigten Metadaten.

Die Bewertungsregeln stammen aus dem Kodierschema.

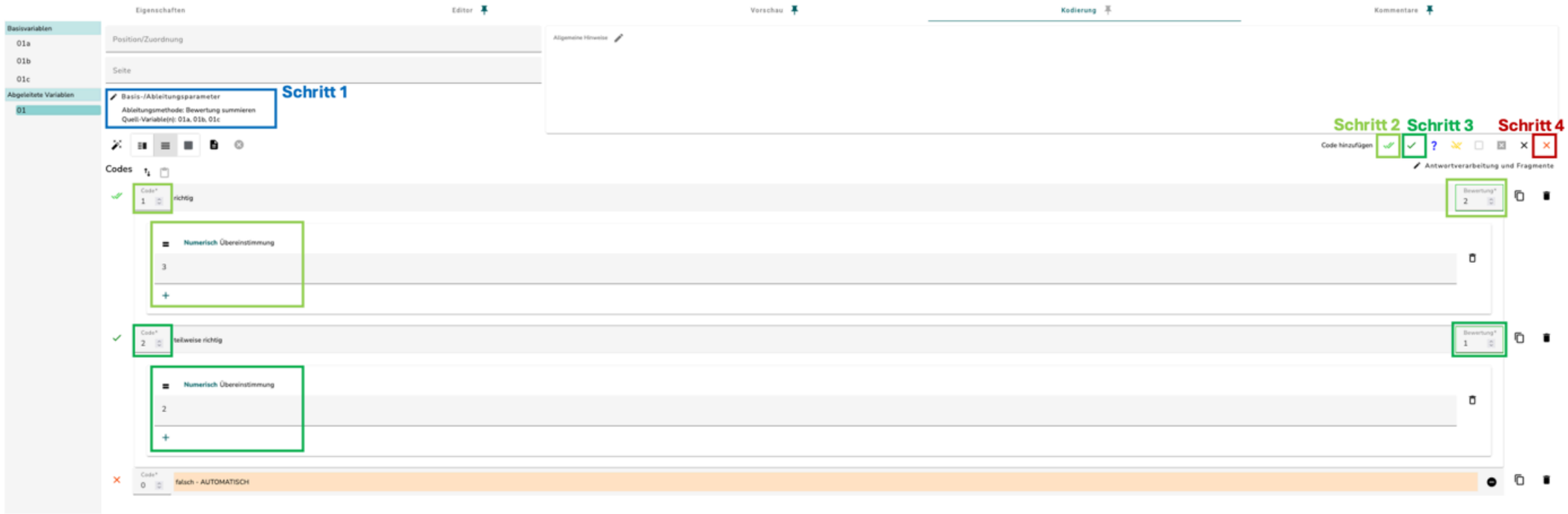
Basisvariablen und abgeleitete Variable



`01a`, `01b`, `01c` speichern einzelne Antworten. `01` wird später aus deren Bewertungen abgeleitet.

Die Basisvariablen und die Ableitung sind links in der IQB-Studio-Ansicht getrennt aufgeführt.

Partial Credit: die abgeleitete Variable 01



Die Ableitung summiert die drei Basisbewertungen. Aus der Summe entsteht die Punktzahl für **ein Item**.

Das Kodierschema in R nachvollziehen

```r
pc_rules <- pc_units |>
  select(unit_key, unit_codes) |>
  unnest(unit_codes) |>
  unnest(variable_codes) |>
  select(variable_id, variable_source_type, code_id, code_score)
pc_rules
```

```
# A tibble: 5 × 4
  variable_id variable_source_type code_id code_score
  <chr>       <chr>                  <int>      <dbl>
1 01a         BASE                       1          1
2 01a         BASE                       0          0
3 01          SUM_SCORE                  1          2
4 01          SUM_SCORE                  2          1
5 01          SUM_SCORE                  0          0
```

01a wird direkt bewertet; 01 erhält aus der Summe 0, 1 oder 2 Punkte. Code und Score sind unterschiedliche Angaben.

Mit eatAutoCode neu kodieren

`code_responses()` verwendet das Kodierschema und berechnet auch die abgeleitete Variable.

```r
pc_coded <- code_responses(pc_responses, pc_units, prepare = TRUE)
pc_coded |>
  filter(login_name == "D1") |>
  select(variable_id, value, code_id, code_score, code_status)
```

```
# A tibble: 4 × 5
  variable_id value code_id code_score code_status
  <chr>       <chr>   <int>      <int> <chr>
1 01a         21          1          1 CODING_COMPLETE
2 01b         112         1          1 CODING_COMPLETE
3 01c         1102       11          1 CODING_COMPLETE
4 01          3           1          2 CODING_COMPLETE
```

Skript: 02_daten_verstehen.R

Welche Variable kommt in die Skalierung?

Die Item-Verknüpfung wählt 01 aus. 01a, 01b und 01c bleiben als Ausgangsvariablen nachvollziehbar.

```r
pc_items <- add_item_id(pc_coded, pc_units)
pc_items <- filter(pc_items, !is.na(item_id))
select(pc_items, login_name, item_id, value, code_id, code_score)
```

```
# A tibble: 4 × 5
  login_name item_id    value code_id code_score
  <chr>      <chr>      <chr>   <int>      <int>
1 D1         RECHNEN_01 3           1          2
2 D2         RECHNEN_01 2           2          1
3 D3         RECHNEN_01 1           0          0
4 D4         RECHNEN_01 0           0          0
```

Skript: 02_daten_verstehen.R

Beispiel 2: die vollständige Aufbereitung

Die synthetische Lesestudie bildet die Struktur von API-Objekten nach:

- 40 vorgesehene Personen, 6 Units und 8 Items.
- Zwei Booklets mit teilweise gemeinsamen Aufgaben.
- Automatische, abgeleitete und manuelle Kodierungen.
- Teilpunkte und gezielt fehlende Antworten.

Einlesen → Kodieren → Design ergänzen → Personen und Items auswählen → Kennwerte → Export

Skript: 03_datenaufbereitung.R

R-Projekt öffnen und Pakete laden

Das Arbeitsverzeichnis ist der entpackte Projektordner.

```r
library(eatPrepTBA) # lädt eatAutoCode mit; nicht separat laden
library(dplyr)
library(tidyr)
person_columns <- c("group_id", "login_name", "login_code")

dir.create("outputs", showWarnings = FALSE)
dir.create("checkpoints", showWarnings = FALSE)
```

Alle Pfade beziehen sich auf diesen Ordner. Die Ausgaben landen in `outputs/`.

Zum Skript: 03_datenaufbereitung.R

Synthetische Studiendaten einlesen

```r
units <- readRDS("data/units.rds")
design <- readRDS("data/design.rds")
responses <- read_responses("data/Responses.csv")

# Nach einem API-Abruf alternativ:
# responses <- readRDS("data_eigen/responses.rds")

count(responses, booklet_id)
saveRDS(responses, "checkpoints/01_responses.rds")
```

```
# A tibble: 4 × 4
  login_name booklet_id unit_key response_progress
  <chr>      <chr>      <chr>    <chr>
1 P001       B1         U1       some
2 P001       B1         U2       some
3 P001       B1         U3       some
4 P001       B1         U4       some
```

Eine Zeile gehört zu einer Person und Unit. Die einzelnen Antworten stehen noch im JSON.

Units und Booklets der Lesestudie

Ausschnitte aus den mitgelieferten synthetischen R-Tabellen:

```
# A tibble: 6 × 2
  unit_key unit_label
  <chr>    <chr>
1 U1       Bibliothek
2 U2       Aushang
3 U3       Zwei Textbelege
4 U4       Begruendung
5 U5       Busfahrplan
6 U6       Museum
```

```
# A tibble: 5 × 3
  booklet_id unit_booklet_no unit_key
  <chr>                <dbl> <chr>
1 B1                       1 U1
2 B1                       2 U2
3 B1                       3 U3
4 B1                       4 U4
5 B1                       5 U5
```

Eine Designzeile bedeutet: diese Variable ist vorgesehen

Das Design enthält auch Einträge, zu denen später keine Antwort gespeichert wird.

```
# A tibble: 6 × 4
  login_name booklet_id unit_key variable_id
  <chr>      <chr>      <chr>    <chr>
1 P001       B1         U1       Q1
2 P001       B1         U1       Q2
3 P001       B1         U2       Q1
4 P001       B1         U3       Q1
5 P001       B1         U3       Q2
6 P001       B1         U3       TOTAL
```

B1 enthält U1–U5; B2 enthält U1–U4 und U6. Für Personen mit B1 ist U6 deshalb nicht vorgesehen.

Antworten mit dem Testdesign abgleichen

```r
response_keys <- c("group_id", "login_name", "login_code",
                   "booklet_id", "unit_key")

unknown_units <- anti_join(responses, units, by = "unit_key")
outside_design <- anti_join(responses, design, by = response_keys)
missing_rows <- anti_join(design, responses, by = response_keys)
```

```
# A tibble: 3 × 2
  Fall                      Anzahl
  <chr>                      <int>
1 Antwort ohne Unit              0
2 Antwort außerhalb Design       0
3 Designzeile ohne Antwort      11
```

Vorgesehene Einträge ohne Antwort werden später ergänzt. Antworten ohne passende Unit oder Designzuordnung erfordern einen Abgleich der Datenstände.

code_responses(): Eingang und Ausgabe

Argument	Bedeutung
`responses`	Ursprüngliche Antworten aus `get_responses()` oder `read_responses()`; noch nicht entpackt
`units`	Passende Aufgabeninformationen mit `coding_scheme` und `unit_variables`
`prepare = FALSE`	Kodiert neu; `value` bleibt eine Listenspalte
`prepare = TRUE`	Entpackt anschließend `value` und ergänzt unter anderem `variable_source_type` und `code_type`

Im folgenden Ablauf: **`prepare = TRUE`**.

`prepare_coded()` liest dagegen bereits gespeicherte Kodierungen aus; es führt keine neue Kodierung durch.

code_responses(): weitere Argumente

Argument	Bedeutung
`codes_manual = NULL`	Optional: Tabelle mit manuellen Codes. Scores stammen aus dem Kodierschema; abhängige Variablen werden neu berechnet.
`missings = NULL`	Optional: Zuordnung manueller Missing-Codes mit `code_id`, `code_status`, `code_score`, `code_type`. Ohne Angabe gelten Paketvoreinstellungen.
`overwrite = FALSE`	Verwendet vorhandene `unit_codes`. Bei `TRUE` werden diese Metadaten erneut aus dem Kodierschema aufgebaut.

`missings` betrifft hier das Einsetzen manueller Codes. Die endgültige Missing-Einordnung folgt mit `complete_design()`.

`code_responses()`: vollständige Hilfe

Automatisch kodieren

```r
coded_auto <- code_responses(responses, units, prepare = TRUE)
count(coded_auto, unit_key, code_status)
saveRDS(coded_auto, "checkpoints/02_coded_auto.rds")
```

```
# A tibble: 4 × 5
  unit_key variable_id code_id code_score code_status
  <chr>    <chr>          <int>      <dbl> <chr>
1 U3       Q1                 0          0 CODING_COMPLETE
2 U3       Q2                 0          0 CODING_COMPLETE
3 U3       TOTAL              0          0 CODING_COMPLETE
4 U4       TEXT              NA         NA CODING_INCOMPLETE
```

U3/TOTAL ist bereits abgeleitet; U4/TEXT benötigt eine manuelle Bewertung.

Manuelle Codes: vollständige Eingabetabelle

```r
codes_manual <- readr::read_csv("data/codes_manual.csv",
  col_types = readr::cols(.default = "c", code_id = "i"))
```

```r
head(codes_manual, 3) |> print(width = 90) # sollte 7 Spalten zeigen
```

```
# A tibble: 3 × 7
  group_id login_name login_code booklet_id unit_key variable_id code_id
  <chr>    <chr>      <chr>      <chr>      <chr>    <chr>          <int>
1 WS       P001       SYN001     B1         U4       TEXT               1
2 WS       P002       SYN002     B1         U4       TEXT               2
3 WS       P003       SYN003     B1         U4       TEXT               0
```

Die sechs Kennungen ordnen den **`code_id`** einer Antwort zu. Die Punktzahl wird aus dem Kodierschema übernommen.

Manuelle Codes einsetzen

```r
coded <- code_responses(responses, units, prepare = TRUE,
                        codes_manual = codes_manual)
count(coded, code_status, code_type)
saveRDS(coded, "checkpoints/03_coded.rds")
```

```
# A tibble: 4 × 5
  login_name variable_id code_id code_score code_status
  <chr>      <chr>          <int>      <dbl> <chr>
1 P001       TEXT               1        0.5 CODING_COMPLETE
2 P002       TEXT               2        1   CODING_COMPLETE
3 P003       TEXT               0        0   CODING_COMPLETE
4 P004       TEXT               1        0.5 CODING_COMPLETE
```

Die ursprünglichen Antworten werden erneut übergeben. Ohne manuelle Kodierung genügt `coded <- coded_auto`.

complete_design(): Daten und Zuordnung

Argument	Bedeutung
`coded`	Kodierte Antworten mit `prepare = TRUE`
`units`	Aufgabeninformationen einschließlich Kodierschema und Positionen innerhalb der Unit
`design`	Alle vorgesehenen Personen, Booklets und Variablen mit ihrer Reihenfolge
`identifiers`	Personenkennungen; Standard: `group_id`, `login_name`, `login_code`
`overwrite = FALSE`	Vorhandene `unit_codes` verwenden; `TRUE` bereitet sie erneut auf
`missings = NULL`	Optional: Tabelle mit `code_id`, `code_status`, `code_score`, `code_type`; legt Missing-Codes und Scores fest

Die endgültige Missing-Art steht in **`code_type`**. Der technische `code_status` bleibt erhalten.

complete_design(): Umgang mit fehlenden Antworten

Argument	Wirkung bei TRUE
`recode_omissions_to_not_reached`	Abschließende Auslassungen innerhalb eines Testlets werden zu Not reached. Innere Auslassungen bleiben bestehen.
`preserve_not_reached`	Vorhandene oder beim Ergänzen erkannte Not-reached-Fälle bleiben erhalten, auch wenn später bearbeitete Variablen folgen.

Beide Optionen sind standardmäßig **FALSE** und unabhängig voneinander.

Die Reihenfolge im Testdesign und die Positionen innerhalb der Unit bestimmen, ob eine fehlende Antwort vor oder nach späterer Bearbeitung liegt.

`complete_design()`: vollständige Hilfe

Design ergänzen und Missings zuweisen

```r
design_coded <- complete_design(coded, units, design,
  recode_omissions_to_not_reached = FALSE, preserve_not_reached = FALSE)

count(design_coded, id_used, code_type)
saveRDS(design_coded, "checkpoints/04_design_coded.rds")
```

```
# A tibble: 3 × 5
  login_name unit_key variable_id code_type           code_score
  <chr>      <chr>    <chr>       <chr>                    <dbl>
1 P002       U2       Q1          MISSING_BY_OMISSION          0
2 P003       U5       Q1          MISSING_NOT_REACHED         NA
3 P003       U5       Q2          MISSING_NOT_REACHED         NA
```

P002/U2 fehlt vor späterer Bearbeitung: Auslassung. P003/U5 fehlt am Testletende: nicht erreicht.

Missing-Art und Punktwert auseinanderhalten

Art	Code	Standardscore nach `complete_design()`
Keine Kodierung (`NO_CODING`)	−93	NA
Not reached	−96	NA
Coding impossible	−97	NA
Invalid response	−98	0
By omission	−99	0
By design	außerhalb des vorgesehenen Designs	NA beim Umformen

Das sind Paketvoreinstellungen. Studienspezifische Regeln ausdrücklich festlegen.

Eine andere Missing-Regel vergleichen

```r
omissions_reclassified <- complete_design(coded, units, design,
  recode_omissions_to_not_reached = TRUE, preserve_not_reached = TRUE)

count(omissions_reclassified, id_used, code_type)
```

```
# A tibble: 4 × 5
  Regel       login_name unit_key code_type           code_score
  <chr>       <chr>      <chr>    <chr>                    <dbl>
1 Standard    P006       U5       MISSING_BY_OMISSION          0
2 Standard    P006       U5       MISSING_BY_OMISSION          0
3 Alternative P006       U5       MISSING_NOT_REACHED         NA
4 Alternative P006       U5       MISSING_NOT_REACHED         NA
```

Bei P006/U5 werden abschließende Auslassungen zu Not reached. Der Standardexport verwendet weiterhin `design_coded`.

Personen für die Auswertung auswählen

```r
analysis_data <- filter(design_coded, id_used)
count(design_coded, id_used)
```

```
# A tibble: 4 × 5
  login_name unit_key variable_id id_used code_score
  <chr>      <chr>    <chr>       <lgl>        <dbl>
1 P001       U1       Q1          TRUE             0
2 P001       U1       Q2          TRUE             1
3 P001       U2       Q1          TRUE             0
4 P001       U3       Q1          TRUE             0
```

```
# A tibble: 1 × 1
  n_personen
       <int>
1         39
```

`id_used` bedeutet: mindestens ein gespeicherter Kodierstatus liegt vor. P040 ohne Antworten entfällt; es bleiben 39 Personen.

Item-Verknüpfungen ergänzen

Variablen ohne Item-ID ansehen und nur verknüpfte Items in den Skalierungsdatensatz aufnehmen.

```r
design_items <- add_item_id(design_coded, units)
item_data <- add_item_id(analysis_data, units)

unmapped <- item_data |>
  filter(is.na(item_id) | trimws(item_id) == "") |>
  distinct(unit_key, variable_id)
print(unmapped) # sollte U3/Q1 und U3/Q2 zeigen

item_data <- filter(item_data, !is.na(item_id), trimws(item_id) != "")
item_data <- mutate(item_data, item = paste0(unit_key, "__", item_id))
```

`unit_key__item_id` unterscheidet gleich benannte Items aus verschiedenen Units.

Zum Skript: 03_datenaufbereitung.R

Aus Variablen werden Auswertungsitems

```r
item_data |>
  filter(login_name == "P001", unit_key %in% c("U3", "U4")) |>
  select(unit_key, variable_id, item_id, item, code_score) |>
  print()
```

```
# A tibble: 2 × 5
  unit_key variable_id item_id  item          code_score
  <chr>    <chr>       <chr>    <chr>              <dbl>
1 U3       TOTAL       U3_TOTAL U3__U3_TOTAL         0
2 U4       TEXT        U4_TEXT  U4__U4_TEXT          0.5
```

U3/TOTAL wird ein Item; die Basisvariablen Q1 und Q2 erhalten keine zusätzlichen Spalten im Skalierungsdatensatz.

Offene Itemkodierungen erkennen

```r
unresolved <- item_data |>
  filter(code_status %in% c("UNSET", "CODING_INCOMPLETE", "DERIVE_PENDING",
                            "CODING_ERROR", "DERIVE_ERROR",
                            "INTENDED_INCOMPLETE") |
         code_type %in% c("NO_CODING", "CODING_INCOMPLETE", "DERIVE_PENDING",
                          "INTENDED_INCOMPLETE"))
stopifnot(nrow(unresolved) == 0) # sollte ohne Fehler durchlaufen
```

```
# A tibble: 0 × 4
# ℹ 4 variables: login_name <chr>, item <chr>, code_status <chr>, code_type <chr>
```

Hier ist die Tabelle leer. Insbesondere `NO_CODING` (−93) bleibt vor einem Export für zu bewertende Items zu klären; technische Hilfsvariablen können unkodiert bleiben.

Missing-Anteile je Person

```r
missing_summary <- summarise_missings(design_items)
missing_summary_used <- summarise_missings(design_items, used_only = TRUE)

select(missing_summary, all_of(person_columns), id_used, n_items,
       n_not_reached, n_by_omission, prop_missing)
readr::write_csv(missing_summary, "outputs/missing_summary.csv")
```

```
# A tibble: 4 × 5
  login_name n_items n_not_reached n_by_omission prop_missing
  <chr>        <int>         <int>         <int>        <dbl>
1 P001             7             0             0        0
2 P002             7             0             1        0.143
3 P003             7             2             0        0.286
4 P004             7             0             0        0.143
```

`n_items` zählt vorgesehene Items. Nicht vorgesehene Aufgaben gehen nicht als fehlende Antworten in den Nenner ein.

Psychometrische Kennwerte berechnen

Die Bereichszuordnung muss zur eigenen Studie passen.

```r
domains <- distinct(units, unit_key)
domains$domain <- "Lesen"

psychometrics <- evaluate_psychometrics(analysis_data, units, domains = domains)
psychometrics <- add_item_id(psychometrics, units)

code_stats <- psychometrics |>
  select(item_id, unit_key, variable_id, code_id, code_score,
         code_n, code_p_valid, code_pbc) |>
  distinct()
readr::write_csv(code_stats, "outputs/psychometrics.csv")
```

Die Kennwerte der synthetischen Studie zeigen, wie die Ausgabe zu lesen ist.

Zum Skript: 03_datenaufbereitung.R

Antwortkategorie, Code und Score

MC-Item U1_Q1 aus der synthetischen Lesestudie

Antwortkategorie	Code	Score
A · richtige Auswahl	1	1
B · Distraktor	0	0
C · Distraktor	0	0
Ungültige Antwort	−98	0

Kategorien unterscheiden die gewählten Antworten.

Codes fassen die Bewertung zusammen.

Scores geben die Punktzahl an.

Antwortkategorien: Welche Auswahl tritt wie oft auf?

```r
category_stats <- psychometrics |>
  filter(item_id == "U1_Q1", !is.na(category_id)) |>
  select(category_id, code_id, category_n, category_p_valid, category_pbc)
print(category_stats) # sollte A: 21, B: 5 und C: 12 Antworten zeigen
```

```
# A tibble: 3 × 5
  category_id code_id category_n category_p_valid category_pbc
  <chr>         <dbl>      <int>            <dbl>        <dbl>
1 B                 0          5            0.128      0.00492
2 C                 0         12            0.308     -0.338
3 A                 1         21            0.538      0.312
```

B wurde fünfmal gewählt, C zwölfmal. Beide erhalten Code 0; ihre Häufigkeiten und Zusammenhänge mit dem Bereichsscore bleiben dennoch unterscheidbar.

Codes: Welche Bewertungen treten wie oft auf?

```r
code_stats_mc <- psychometrics |>
  filter(item_id == "U1_Q1") |>
  select(code_id, code_score, code_n, code_p_valid, code_pbc) |>
  distinct()
print(code_stats_mc) # sollte für Code 0 insgesamt 17 Antworten zeigen
```

```
# A tibble: 3 × 5
  code_id code_score code_n code_p_valid code_pbc
    <dbl>      <dbl>  <int>        <dbl>    <dbl>
1     -98          0      1       0.0256 -0.00871
2       0          0     17       0.436  -0.308
3       1          1     21       0.538   0.309
```

Code 0 umfasst B und C: **5 + 12 = 17 Antworten**. Die ungültige Antwort bleibt als Code −98 erkennbar. `distinct()` entfernt wiederholte Codezeilen aus der gemeinsamen Kategorie- und Codeausgabe.

Scores: Welche Punktzahlen gehen in die Skalierung?

```r
score_stats <- item_data |>
  filter(item_id == "U1_Q1") |>
  count(code_score, name = "score_n") |>
  mutate(score_p = score_n / sum(score_n))
print(score_stats) # sollte 18 Personen mit 0 und 21 mit 1 Punkt zeigen
```

```
# A tibble: 2 × 3
  code_score score_n score_p
       <dbl>   <int>   <dbl>
1          0      18   0.462
2          1      21   0.538
```

Score 0: 17 falsche + 1 ungültige Antwort = 18 Personen.

`evaluate_psychometrics()` liefert Kategorie- und Codekennwerte. Die Scorehäufigkeiten werden hier direkt aus den Person-Item-Zeilen berechnet.

Den Skalierungsdatensatz erstellen

```r
scaling_data <- item_data |>
  select(group_id, login_name, login_code, item, code_score) |>
  pivot_wider(names_from = item, values_from = code_score) |>
  arrange(group_id, login_name, login_code) |>
  mutate(person_id = row_number(), .before = 1)
```

```
# A tibble: 4 × 5
  person_id login_name U1__U1_Q1 U3__U3_TOTAL U5__U5_Q1
      <int> <chr>           <dbl>        <dbl>     <dbl>
1         1 P001                0            0         0
2         2 P002                0            1         1
3         3 P003                0            0        NA
4         4 P004                0            0         0
```

Eine Zeile ist eine Person; jede Itemspalte enthält Punkte. `NA` bleibt erhalten, auch bei nicht vorgesehenen Items.

Die Personenkennungen zuordnen

```r
person_key <- select(scaling_data, person_id, group_id, login_name, login_code)
item_columns <- setdiff(names(scaling_data), names(person_key))
persons <- distinct(item_data, group_id, login_name, login_code)
stopifnot(nrow(scaling_data) == nrow(persons))
stopifnot(all(vapply(scaling_data[item_columns], is.numeric, logical(1))))
```

```
# A tibble: 4 × 4
  person_id group_id login_name login_code
      <int> <chr>    <chr>      <chr>
1         1 WS       P001       SYN001
2         2 WS       P002       SYN002
3         3 WS       P003       SYN003
4         4 WS       P004       SYN004
```

`person_key` verbindet `person_id` mit den ursprünglichen Testcenter-Kennungen.

Datensatz, Schlüssel und Long-Daten speichern

```r
readr::write_csv(scaling_data, "outputs/scaling_data.csv")
readr::write_csv(person_key, "outputs/person_key.csv")
saveRDS(analysis_data, "outputs/analysis_long.rds")
saveRDS(design_items, "outputs/design_items.rds")
saveRDS(scaling_data, "checkpoints/05_scaling_data.rds")

missing_counts <- count(design_coded, id_used, code_type)
readr::write_csv(missing_counts, "outputs/missing_counts.csv")
```

```
# A tibble: 5 × 2
  Datei              Bytes
  <chr>              <dbl>
1 scaling_data.csv    1561
2 person_key.csv       734
3 analysis_long.rds   2401
4 design_items.rds    2531
5 missing_counts.csv   219
```

RDS bewahrt die ausführlichen Long-Daten einschließlich Missing-Arten. Fallzahlen, Regeln und Paketversionen stehen im Prüfprotokoll.

3 · Logdaten im Überblick

Ereignisse → Prüfhinweise → Zeiten und Fokusverluste

Das lokale Beispiel enthält zwei Sessions mit 15 Ereignissen.

`R/04_logs.R` zeigt den gesamten Ablauf mit Testcenter-Logdaten.

Logdaten einlesen und einzelne Ereignisse ansehen

```r
logs <- read_logs("data/Logs.csv")
# Alternativ nach API-Abruf: logs <- readRDS("data_eigen/logs.rds")

# Ältere Exporte haben keinen getrennten Unit-Alias.
if (!"unit_alias" %in% names(logs)) logs$unit_alias <- logs$unit_key

count(logs, group_id, login_name, booklet_id) # P001: 9, P002: 6 Ereignisse
```

```
# A tibble: 4 × 4
  login_name unit_key ts            log_entry
  <chr>      <chr>    <chr>         <chr>
1 P001       U1       1780000000100 "PLAYER = LOADING"
2 P001       U1       1780000000450 "PLAYER = RUNNING"
3 P001       U1       1780000004000 "FOCUS : \"HAS_NOT\""
4 P001       U1       1780000006500 "FOCUS : \"HAS\""
```

`ts` enthält den Zeitstempel, `log_entry` das protokollierte Ereignis.

Welche Ereignistypen sind vorhanden?

```
1 inventory <- summarise_log_inventory(logs)
```

```
# A tibble: 5 × 4
  log_type            n known_log_type supported_parser
  <chr>           <int> <lgl>          <lgl>
1 FOCUS               3 TRUE           TRUE
2 PLAYER              3 TRUE           TRUE
3 CURRENT_PAGE_NR     2 TRUE           TRUE
4 PAGE_COUNT          2 TRUE           TRUE
5 CONNECTION          1 TRUE           TRUE
```

`known_log_type` kennzeichnet bekannte Ereignistypen; `supported_parser` zeigt, ob ein Parser für ihre Aufbereitung verfügbar ist.

Hilfe: summarise_log_inventory()

Auffälligkeiten vor der Zeitinterpretation prüfen

```r
anomalies <- detect_log_anomalies(logs)
qc <- summarise_log_qc(logs, anomalies = anomalies)

anomalies |>
  select(login_name, booklet_id, anomaly_code, severity, evidence)
```

```
# A tibble: 2 × 4
  login_name booklet_id log_qc_flag n_anomalies
  <chr>      <chr>      <chr>             <int>
1 P001       B1         ok                    0
2 P002       B1         critical              7
```

P002 meldet einen Playerfehler ohne erfolgreichen Start. `anomalies` enthält die einzelnen Hinweise; `qc` fasst sie je Session zusammen. Ein QC-Flag begründet eine Prüfung, keinen automatischen Ausschluss.

Beobachtete Unit- und Ladezeiten in Sekunden

```r
unit_times <- estimate_unit_times(logs, include_environment = FALSE)

# Millisekunden in Sekunden umrechnen:
times <- unit_times |>
  transmute(group_id, login_name, login_code, booklet_id, unit_key,
            unit_n_play, unit_time_s = unit_time / 1000,
            unit_loadtime_s = unit_loadtime / 1000,
            n_run_no_load, n_failed_loadings)
```

```
# A tibble: 1 × 4
  login_name unit_key unit_time_s unit_loadtime_s
  <chr>      <chr>          <dbl>           <dbl>
1 P001       U1              11.6            0.35
```

Die Zeit reicht von `RUNNING` bis zum nächsten Unit-Start oder beobachteten Ende. Ohne `RUNNING` entsteht keine Zeitzeile. Fokusverluste sind enthalten.

Abgeschlossene und offene Fokusverluste unterscheiden

```r
focus <- summarise_log_focus(logs)
focus_summary <- focus |>
  transmute(group_id, login_name, login_code, booklet_id,
            n_focus_lost, n_focus_regained,
            focus_lost_s = total_focus_lost_time / 1000,
            max_focus_lost_s = max_focus_lost_time / 1000,
            has_unresolved_focus_loss)
```

```
# A tibble: 2 × 4
  login_name n_focus_lost focus_lost_s has_unresolved_focus_loss
  <chr>             <int>        <dbl> <lgl>
1 P001                  1          2.5 FALSE
2 P002                  1          0   TRUE
```

Nur HAS_NOT → HAS ergibt eine abgeschlossene Dauer. Bei P002 bleibt der Fokusverlust offen: 0 Sekunden bedeutet hier keine gemessene Dauer.

Sessionübersicht und Logergebnisse speichern

```r
session_keys <- c("group_id", "login_name", "login_code", "booklet_id")
session_overview <- qc |>
  select(all_of(session_keys), log_qc_flag) |>
  left_join(focus_summary, by = session_keys)

readr::write_csv(inventory, "outputs/log_inventory.csv")
readr::write_csv(anomalies, "outputs/log_anomalies.csv")
readr::write_csv(times, "outputs/log_unit_times.csv")
readr::write_csv(session_overview, "outputs/log_sessions.csv")
saveRDS(list(inventory = inventory, anomalies = anomalies, qc = qc,
             unit_times = unit_times, focus = focus), "outputs/log_qc.rds")
```

```
# A tibble: 2 × 4
  login_name log_qc_flag n_focus_lost focus_lost_s
  <chr>      <chr>              <int>        <dbl>
1 P001       ok                     1          2.5
2 P002       critical               1          0
```

Fokusdauer gilt pro Session, Bearbeitungszeit pro Unit. Die Session-Fokusdauer lässt sich daher nicht von jeder Unit-Zeit abziehen.

4 · Codebücher und Kodiermanuale

Kodierschemata dokumentieren und für die Auswertung oder Weitergabe aufbereiten.

Ein **Codebuch** macht Variablen, Codes, Beschreibungen und Scores als Tabelle lesbar. Ein **Kodiermanual** ergänzt Anweisungen für die Bewertung von Antworten.

1. **Eigene Studie:** Codebuch aus den Kodierschemata in IQB-Studio erstellen; als Excel oder DOCX bereitstellen.
2. **Ohne Zugang üben:** Eine vorbereitete synthetische Tabelle verwenden, Units auswählen und exportieren.

Mit eigenem Zugang: IQB-Studio vorbereiten

```r
studio_version <- "21.0.0" # Version der eigenen Anmeldeseite übernehmen
studio_workspace_id <- 1300 # Beispiel: eigene Workspace-ID einsetzen
selected_units <- c("UNIT-1", "UNIT-2") # Eigene Unit-Namen; NULL für alle
missing_profile <- "IQB-Standard" # Eigenen Profilnamen prüfen; NULL ohne Profil
export_path <- "outputs/codebooks_studio"
dir.create(export_path, recursive = TRUE, showWarnings = FALSE)
```

```r
login <- login_studio(app_version = studio_version, keyring = TRUE)
workspace <- access_workspace(login, ws_id = studio_workspace_id)
```

Vorlage für eigene Daten: Version, Workspace und Unit-Auswahl anpassen. Die Anmeldung verwendet www.iqb-studio.de und den lokalen Passwortspeicher.

Codebuch aus IQB-Studio abrufen

```r
cb_studio <- prepare_codebook(workspace, unit_keys = selected_units,
                              missings_profile = missing_profile,
                              only_coded = TRUE, show_score = TRUE,
                              derived = TRUE, manual = TRUE, closed = TRUE)
names(cb_studio)
head(cb_studio)
writexl::write_xlsx(cb_studio, file.path(export_path, "codebook.xlsx"))
```

`only_coded` wählt Variablen mit Codes. `derived`, `manual` und `closed` nehmen abgeleitete, manuell kodierte und geschlossen kodierbare Variablen auf.

`names()` und `head()` zeigen die tatsächlich zurückgegebenen Spalten. `show_score` fordert Scoreangaben an; eine separate numerische `code_score`-Spalte ist dabei nicht garantiert.

Hilfe: prepare_codebook()

DOCX-Kodiermanual direkt aus IQB-Studio herunterladen

```
download_codebook(workspace, path = export_path, unit_keys = selected_units,
                  format = "docx", general_instructions = TRUE,
                  missings_profile = missing_profile, show_score = TRUE)
```

`general_instructions = TRUE` übernimmt allgemeine Kodieranweisungen. `missings_profile` ergänzt das in IQB-Studio ausgewählte Missing-Profil.

Die DOCX-Datei wird unter `export_path` gespeichert. Ihr Name stammt aus dem Workspace-Namen. Der Download benötigt den eigenen IQB-Studio-Zugang.

Hilfe: download_codebook()

Ohne Zugang: vorbereitete Codebuchtabelle laden

```r
cb <- readr::read_csv("data/codebook.csv", show_col_types = FALSE,
                      col_types = readr::cols(code_id = readr::col_character()))
count(cb, unit_key, variable_id) # 10 Variablen aus 6 Units
```

```r
cb |>
  select(unit_key, variable_id, code_id, code_label, code_score) |>
  head(4) |> print()
```

```
# A tibble: 4 × 5
  unit_key variable_id code_id code_label          code_score
  <chr>    <chr>       <chr>   <chr>                    <dbl>
1 U1       Q1          1       Richtige Auswahl A           1
2 U1       Q1          0       Andere Auswahl               0
3 U1       Q2          1       Richtige Auswahl A           1
4 U1       Q2          0       Andere Auswahl               0
```

Die synthetische Datei ersetzt hier den Abruf aus IQB-Studio. Das Einlesen ist nur der Einstieg in die lokale Übung.

Units auswählen und Code von Score unterscheiden

```r
selected_cb <- cb |>
  filter(unit_key %in% c("U3", "U4")) |>
  select(unit_key, variable_id, code_id, code_label, code_score)
```

```r
selected_cb |>
  filter(unit_key == "U4", variable_id == "TEXT") |>
  print() # Code 2: Score 1; Code 1: Score 0.5
```

```
# A tibble: 3 × 5
  unit_key variable_id code_id code_label                        code_score
  <chr>    <chr>       <chr>   <chr>                                  <dbl>
1 U4       TEXT        2       Begruendung nennt den Textbeleg          1
2 U4       TEXT        1       Passende Aussage ohne Textbeleg          0.5
3 U4       TEXT        0       Unzutreffende Begruendung                0
```

Bei U4/TEXT bedeutet Code 2 einen Punkt, Code 1 einen halben Punkt. Die Code-ID ist keine Punktzahl.

Codebuch als Excel und CSV speichern

```r
writexl::write_xlsx(list(Codebuch = cb, Auswahl = selected_cb),
                    "outputs/codebook.xlsx")
readr::write_csv(selected_cb, "outputs/codebook.csv")
```

```r
codebook_files <- c("outputs/codebook.xlsx", "outputs/codebook.csv")
tibble(Datei = basename(codebook_files),
       Bytes = file.info(codebook_files)$size) |> print()
```

```
# A tibble: 2 × 2
  Datei         Bytes
  <chr>         <dbl>
1 codebook.xlsx  6818
2 codebook.csv    407
```

Excel enthält die Blätter `Codebuch` und `Auswahl`; die CSV enthält die ausgewählten Units.

Auf eigene Daten übertragen

1. **Daten beziehen:** In `01_datenabruf.R` Zugänge und Workspace eintragen oder vorhandene Exporte einlesen.
2. **Aufbereitung anpassen:** In `03_datenaufbereitung.R` Pfade, manuelle Codes, Missing-Regeln und Bereichszuordnung festlegen.
3. **Ergebnisse sichern:** Datensatz, Long-Daten, Personenschlüssel und Prüfprotokoll gemeinsam aufbewahren.

Workshop-Repository · Handout · Nachschlagen

Downloadpaket mit R-Skripten und synthetischen Daten

Quellen und Paketstand

- eatPrepTBA **0.9.8.9042** (`ea782df`), eatAutoCode **0.5.0.9008**; geprüft am 24.10.2026.
- Standardworkflow und API-Einführung.
- Logdaten und Codebücher.
- Codegestaltung: eatPrep-Workshop 2024.
- Partial Credit in IQB-Studio: Original-Screenshots, dazu passendes synthetisches Rechenbeispiel.
- Original-Paketlogo; IQB-Styleguide und Präsentationsvorlage.

Lehrdaten synthetisch; Quellen und technische Prüfung im Materialpaket.

Vielen Dank für eure Aufmerksamkeit!

Gibt es Fragen?